

Lecture 02 - The Rendering Pipeline

Primary source: `course_text_parts/03_lectures/02_rendering-pipeline.txt`

Supporting course sources:

- ``course_text_parts/01_course_overview.txt``
- ``course_text_parts/05_opengl_starter_project.txt``

A beginner-friendly textbook chapter for a student who missed the lecture. It teaches the dependency chain first, defines vocabulary before relying on it, and uses concrete OpenGL examples only after the underlying idea is clear.

How to use this chapter

Read it in order. The chapter intentionally defines the terms needed later - coordinate system, clip space, shader, interpolation, fragment, framebuffer, attachment, and depth buffer - before using them as if they were obvious.

This is still grounded in the lecture material rather than replacing it. The goal is to make the slides studyable: the slides provide the official vocabulary and diagrams, while this reader explains the causal chain between them.

Source anchors

- Lecture 2 slides 2 and 5-9: object-based rendering, geometry stage, rasterization stage, fragments, framebuffer.
- Lecture 2 slides 12-17: OpenGL as a stateful graphics API, context creation, geometry upload, rendering.
- Lecture 2 slides 22-33: immediate mode, VBOs, VAOs, `glBufferData`, `glVertexAttribPointer`, `glDrawArrays`, interleaved attributes.
- Lecture 2 slides 35-36 and 46-56: fixed and programmable pipeline stages, shaders, coordinate systems, clipping, perspective division, viewport transform.
- Lecture 2 slides 57-64: fragment shaders, shader programs, varyings, uniforms, fragment shader limits.
- Lecture 2 slides 68-85: fragment tests, depth/stencil/blending, framebuffer structure, clearing, double buffering.
- Course overview and starter project: OpenGL/C++ practice context, GLM/resources, rotating-cube code using a model-view-projection matrix, attributes, uniforms, depth testing, and `glViewport`.

1. The one problem behind the whole chapter

Source focus: 2, 5-6, 35-36, 92

Rendering starts with one mismatch: a scene is described as objects in 3D space, but the display can only show a 2D array of pixel values. The lecture summarizes this as object-based rendering: scene objects are made from 3D models and primitives, primitives become fragments, and only some fragments become pixels.

A useful first definition is a representation. A representation is the form in which information is currently stored. Three vertex positions are one representation of a triangle. A covered set of screen samples is another. A color value in a framebuffer is another. The rendering pipeline is a sequence of representation changes, not a magic black box.

That sequence exists because each stage answers a different question. Object placement asks where things are. Projection asks where a 3D point appears in the image. Rasterization asks which screen samples a projected primitive covers. Fragment tests ask whether a candidate contribution is allowed to affect the final image.

The minimal story is: scene description -> vertex data -> primitives -> projected primitives -> fragments -> fragment tests and operations -> framebuffer pixels. Keep this chain in mind; the OpenGL calls make more sense once you know which representation they prepare or consume.

Concrete example

Imagine a square made from two triangles. Before rendering, it is just numbers in memory. After the geometry work, those numbers describe two triangles in the camera's image. After rasterization, the triangles have produced many fragment candidates. After depth and blending decisions, only selected candidates update the framebuffer.

Check yourself

Why is it more accurate to say that rendering changes representations than to say it simply draws objects?

2. Vocabulary you need before the pipeline details

Source focus: 2, 6-9, 46-58, 68-85

A vertex is one input point of geometry. In this course it usually stores a position, and later it may also store color, a normal vector, or a texture coordinate. A primitive is the simple shape assembled from vertices, such as a point, line, triangle, triangle strip, or patch. Most real-time rendering uses triangles because they are simple, planar, and fast to process.

A coordinate system is a rule for what numbers mean as positions. The same physical corner of a cube can have one coordinate in the cube's local system, another in the world, another from the camera's point of view, and another after projection. This is why the slides emphasize successive coordinate systems before talking about pixels.

A shader is a small program that runs on the GPU at a programmable pipeline stage. A vertex shader runs for each input vertex and must write the final clip-space vertex position to `gl_Position`. A fragment shader runs for each fragment candidate and computes values such as color and depth. Shaders are written in GLSL in the OpenGL part of the course.

A fragment is not yet a pixel. A fragment is a candidate contribution at a framebuffer sample. It may carry color, depth, and interpolated data, but it still has to survive tests such as depth, stencil, scissor, or alpha-related rules before it can update the framebuffer.

The framebuffer is the rendering destination. Think of it as a GPU-side raster with the same grid size for its attached buffers. It can have a color attachment for visible color, a depth buffer for nearest-surface decisions, a stencil buffer for masks, and other attachments depending on the configuration.

Concrete example

If a triangle has red at one vertex and blue at another, those colors are vertex attributes. Rasterization creates fragments between the vertices. Interpolation gives each fragment a blended color value before the fragment shader or later operations decide the final write.

Check yourself

Explain the difference between a vertex, a primitive, a fragment, and a framebuffer pixel.

3. Application, geometry, rasterization, and why the order matters

Source focus: 5-9

The application stage is the CPU-side part of the story. It decides which objects exist, where they are, how they move, how users interact with them, and which objects should be submitted for drawing. The slides list scene modeling, animation paths, collision detection, and occlusion culling here. Those are scene-level decisions, so they happen before the GPU starts turning primitives into fragments.

The geometry stage still works with geometric things: vertices and primitives. It handles transformations, primitive assembly, tessellation, projection, and clipping. It can say that three vertices form one triangle, that a local model should be placed in world space, or that part of a primitive lies outside the visible region.

Rasterization is the point where the representation changes from continuous geometry to a discrete grid. A mathematical triangle covers an area, but the framebuffer contains sample locations. Rasterization determines which samples are covered and produces one fragment candidate per covered sample.

The causal order matters. You cannot depth-test a surface at a screen sample before rasterization has produced a fragment for that sample. You cannot rasterize a triangle before you know where its vertices land on the screen. You cannot project a triangle sensibly before you know where the object and camera are.

Concrete example

For two overlapping triangles, the geometry stage still knows them as triangles. Rasterization turns both into fragment candidates at many of the same sample positions. Only then can the depth test compare which candidate is nearer at each position.

Check yourself

Which stage works with vertices and primitives, and which stage first creates fragment candidates?

4. OpenGL is the control interface, not the scene itself

Source focus: 10-17, 61-63

OpenGL is a platform-independent 2D and 3D graphics API. It does not understand high-level objects like car, lamp, camera, or material in the way a game engine might. Your application creates an OpenGL context, creates GPU resources, sets state, provides shaders and data, and issues drawing commands.

The OpenGL context is important because OpenGL is stateful. The context stores the currently active rendering state: which shader program is used, which vertex array is bound, which buffers and textures are connected, which framebuffer is active, and which tests such as depth or blending are enabled. A draw call depends on that state.

A beginner mistake is to read `glDrawArrays` as if it means draw my object. More precisely, it means use the current OpenGL state to draw a range of vertex records as a particular primitive type. If the wrong program, VAO, framebuffer, or test state is active, the function call can be correct and the image can still be wrong.

The course overview says the practical implementation uses C/C++ and OpenGL. The starter project makes this concrete: it creates a GLFW window, sets a framebuffer-size callback that calls `glViewport`, compiles shaders, links a program, uploads vertex data, sets uniforms, enables depth testing, and then draws with `glDrawArrays`.

Concrete example

Two identical `glDrawArrays(GL_TRIANGLES, 0, 36)` calls can produce different images if one is executed with a cube VAO and a color shader while the other is executed with a different VAO, a different shader program, or depth testing disabled.

Check yourself

Why is the current OpenGL context part of the input to a draw call?

5. Why geometry is uploaded to the GPU

Source focus: 22-33

The lecture contrasts older immediate-mode OpenGL with modern OpenGL. In immediate mode, the application sends vertex commands while drawing. That is easy to read, but the slides point out a performance problem: geometry may be transferred from CPU memory to the GPU on every call, and the transfer can take longer than the drawing.

Modern OpenGL separates preparation from drawing. You put vertex data into buffer objects once, describe the memory layout, and draw from that prepared GPU-side data many times. This matches real-time rendering: the same mesh may appear across many frames while only transformation matrices or uniforms change.

A Vertex Buffer Object, or VBO, stores raw vertex bytes in GPU-accessible memory. A Vertex Array Object, or VAO, stores the vertex input setup used for drawing. The distinction is beginner-critical: the VBO is the data storage; the VAO is the recipe that says how to read that data as vertex attributes.

`glBufferData` creates or replaces buffer storage and optionally copies data from CPU memory. `glVertexAttribPointer` describes one attribute stream: attribute index, component count, component type, normalization behavior, stride, and offset. `glEnableVertexAttribArray` turns that attribute on for the current VAO.

The memory-layout step is not cosmetic. The GPU receives bytes, not your mental model of a vertex. If a vertex record is position followed by color, the position attribute and color attribute use the same stride but different offsets.

Concrete example

For interleaved vertices stored as `[x, y, z, r, g, b]`, each vertex has six floats. Attribute 0 can read position with `size=3`, `stride=6*sizeof(float)`, `offset=0`. Attribute 1 can read color with `size=3`, the same stride, and `offset=3*sizeof(float)`. The VBO bytes are shared; the VAO layout separates the meanings.

Check yourself

What belongs in a VBO, and what does the VAO remember?

6. Draw calls, primitive assembly, and the first pipeline handoff

Source focus: 27-36

Once buffers and attribute layouts are prepared, a draw call selects how a sequence of vertex records should become primitives. With `GL_TRIANGLES`, every group of three vertices becomes an independent triangle. With line or strip modes, the grouping is different. The mode is therefore not decoration; it changes primitive assembly.

Primitive assembly is the first place where individual vertex shader results become larger geometric units. Before assembly, the pipeline sees separate vertices. After assembly, it can reason about lines, triangles, or other primitive forms. This matters because clipping and rasterization operate on primitives, not isolated vertices.

The rendering pipeline then alternates between programmable and fixed-function work. Programmable stages run shader code that you write. Fixed-function stages perform standardized GPU operations such as primitive assembly, clipping, perspective division, viewport transformation, rasterization, and fragment tests.

This division gives you a debugging map. If vertices are uploaded but nothing appears, ask whether the VAO layout matches the shader inputs. If geometry appears in the wrong place, ask whether the vertex shader or matrices are wrong. If the shape appears but colors are wrong, inspect attribute wiring, interpolation, fragment shader output, and blending.

Concrete example

The starter project draws a rotating cube with `glDrawArrays(GL_TRIANGLES, 0, 36)`. The 36 vertex records become 12 triangles. The vertex shader transforms each position with a model-view-projection matrix, and the later stages turn the surviving projected triangles into fragments.

Check yourself

What changes when `GL_TRIANGLES` is replaced by a line or strip primitive mode?

7. Coordinate systems before projection and clip space

Source focus: 46-55

Coordinate systems must be introduced before projection because projection only makes sense once you know what the coordinates are relative to. The slides show the chain local space -> world space -> view space -> clip space -> screen space.

Local space means coordinates relative to one object's own origin. A cube centered at its own origin may have corners near $(-0.5, -0.5, -0.5)$ and $(0.5, 0.5, 0.5)$. World space places that cube among other objects in a shared scene. View space rewrites the same positions from the camera's point of view.

Projection maps view-space geometry into clip space. Orthographic projection keeps parallel lines parallel; perspective projection makes farther objects appear smaller. The lecture later treats projection in more detail, but Lecture 2 needs the purpose: projection prepares geometry for clipping and eventual screen placement.

Clip space is the coordinate form produced by the vertex shader through `gl_Position`. The slides simplify the clip range as -1 to 1 , but the important beginner idea is that clip space is where the GPU can decide what lies inside the viewable volume. Geometry outside that volume is discarded or cut by clipping.

After clipping, fixed-function vertex postprocessing performs perspective division to obtain normalized device coordinates, then the viewport transform maps those normalized coordinates into screen coordinates. `glViewport` defines the rectangle in the pixel grid that receives this mapping. Only after that is geometry ready for rasterization.

Concrete example

A cube vertex can begin as local position $(0.5, 0.5, 0.5)$. A model matrix places it in the world. A view matrix describes it relative to the camera. A projection matrix converts it to clip coordinates. The viewport then maps the resulting normalized position to a pixel-region such as a 640×480 window.

Check yourself

Why should local, world, and view space be understood before clip space?

8. What shaders can and cannot do

Source focus: 37-64

Shaders are the programmable parts of the pipeline. OpenGL shader programs usually combine at least a vertex shader and a fragment shader. The slides show the workflow: create shaders, provide source, compile them, attach them to a program, link the program, and activate it with `glUseProgram`.

A vertex shader runs once for each input vertex. It can read vertex attributes such as position, normal, or texture coordinate, and it can read uniforms such as matrices or light parameters. Its required output is `gl_Position`, the clip-space position. The slides also emphasize limitations: a vertex shader cannot inspect adjacent vertices and cannot generate or discard vertices in this stage.

A uniform is a value supplied by the CPU-side application that stays the same for many shader invocations until the application changes it. The starter project's `uMVP` matrix is a good example: every cube vertex uses the same current model-view-projection matrix for one draw.

A varying is data passed from the vertex shader toward the fragment shader. Between the vertices of a primitive, the rasterizer interpolates those values. Interpolation means computing in-between values. If one vertex outputs red and another outputs blue, fragments between them receive intermediate colors.

A fragment shader runs for fragments, not for whole triangles and not for neighboring pixels. It can compute color and depth, use interpolated data, sample textures, apply fog-like calculations, or discard the current fragment. It cannot generate new fragments, and it normally has no information about adjacent fragments.

Concrete example

In the starter project, the vertex shader reads `aPos` and `aColor`, writes `gl_Position` from `uMVP * vec4(aPos, 1.0)`, and passes `aColor` as `vColor`. The fragment shader receives an interpolated `vColor` and writes `FragColor`. That is the simplest complete example of attributes, a uniform, a varying, and a fragment output working together.

Check yourself

What is the difference between an attribute, a uniform, and a varying?

9. Rasterization and fragments without hand-waving

Source focus: 8-9, 35-36, 55-58

Rasterization is often described as turning triangles into pixels, but that skips a step. More precisely, rasterization turns projected primitives into fragments, and fragments are pixel predecessors. This matches the lecture's wording and prevents a common misunderstanding.

A mathematical triangle is continuous: it contains infinitely many points. A framebuffer is discrete: it has a finite grid of samples. Rasterization asks which sample positions are covered by the projected primitive. For each covered sample, it creates a fragment candidate.

The fragment can carry interpolated data. Depth is interpolated so depth testing can compare candidates. Color, normals, and texture coordinates may be interpolated so the fragment shader can compute a local result. This is why interpolation belongs conceptually between vertex outputs and fragment processing.

The rasterizer specializes in efficient algorithms for primitive types such as lines and triangles. That is why the primitive type chosen at the draw call affects more than just appearance; it determines the shape that rasterization receives.

Concrete example

If a triangle covers only part of a pixel cell, the GPU still evaluates coverage according to its sampling rules. The result is not the abstract triangle itself, but one or more candidate contributions tied to sample locations in the target grid.

Check yourself

Why is a fragment better described as a candidate contribution than as a finished pixel?

10. Fragment tests, depth, stencil, alpha, and blending

Source focus: 68-78

After fragments are generated and processed, they pass through per-fragment tests and operations. The slides list pixel ownership, scissor, alpha, stencil, depth, dithering, and blending. The exact order and availability can depend on OpenGL details, but the conceptual purpose is stable: decide whether and how a fragment updates the framebuffer.

The depth buffer stores a depth value per sample. When depth testing is enabled with `glEnable(GL_DEPTH_TEST)`, the incoming fragment's depth is compared with the depth already stored in the buffer. With a usual nearer-is-better function, a farther fragment loses and does not update the visible color.

The stencil buffer is an image mask buffer. It can be filled indirectly by drawing operations and then used to accept or reject later fragments according to stencil rules. A beginner mental model is a reusable per-pixel mask, although advanced stencil use can be more precise than that phrase suggests.

Alpha is the A component in an RGBA color. The slides describe alpha values as controlling composition: $A=0$ is completely transparent, $A=1$ is fully opaque, and values between them are used for partial opacity. In older OpenGL an alpha test could reject fragments. In modern practice, alpha is usually important for blending and shader-controlled discard.

Blending combines the incoming source fragment with the destination value already in the color buffer. With `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`, a semi-transparent source contributes according to its alpha while the existing destination still contributes according to the remaining weight.

Concrete example

Suppose a red fragment with alpha 0.25 is drawn over a blue destination pixel using ordinary alpha blending. The final color is mostly blue with some red. But if the red fragment fails the depth test first, blending never happens for that fragment.

Check yourself

Name two different reasons a fragment shader's color output might not become the final visible color.

11. Framebuffer, attachments, viewport, clearing, and double buffering

Source focus: 79-85, 87-91

The framebuffer is the final rendering target for this part of the course. The slides call it a context-bound raster with a resolution and structure specified by the application and physically realized on the GPU. It is logically divided into buffers such as color memory, stencil memory, and depth memory.

An attachment is one buffer connected to a framebuffer for a particular purpose. A color attachment stores RGBA color values. A depth attachment stores depth values. A stencil attachment stores mask-like stencil values. The slides note that all buffers of a framebuffer have the same grid size, while their number of bitplanes can differ.

The viewport is the rectangular region of the pixel grid to which normalized positions are mapped. The starter project updates it in a framebuffer-size callback with `glViewport(0, 0, width, height)`. That is why resizing a window is not only a UI event; it changes the mapping from normalized coordinates to screen coordinates.

Clearing initializes framebuffer attachments before drawing a new frame or pass. `glClearColor` sets the color used when clearing the color buffer, and `glClear(GL_COLOR_BUFFER_BIT)` clears that attachment. The starter project clears both color and depth with `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)` so stale colors and stale depth values do not pollute the next frame.

Double buffering uses two color buffers to avoid flicker. The application renders into the back buffer while the front buffer remains visible. When the frame is ready, `glfwSwapBuffers` presents the finished back buffer. This explains why drawing and displaying are related but not the same moment.

Concrete example

If you forget to clear the depth buffer between frames, a new frame may be tested against old depth values. The fragment shader may run correctly, but fragments can still disappear because they lose to stale depth information.

Check yourself

What is the difference between a color attachment and a depth attachment?

12. One triangle from CPU data to visible pixels

Source focus: 2, 17, 22-36, 46-85, 92

Start on the CPU with vertex records, for example `[x, y, z, r, g, b]` for each corner of a triangle. The application creates a VBO and uploads those bytes with `glBufferData`. It creates or binds a VAO and describes position and color attributes with `glVertexAttribPointer` and `glEnableVertexAttribArray`.

The application compiles and links a shader program, binds the program and VAO, sets relevant uniforms such as transformation matrices, chooses pipeline state such as depth testing or blending, and calls `glDrawArrays` with a primitive mode such as `GL_TRIANGLES`.

The vertex shader runs once per input vertex. It reads attributes, uses uniforms, and writes `gl_Position` in clip space. Fixed-function postprocessing clips geometry outside the view volume, performs perspective division, and applies the viewport transform to produce screen-space geometry.

Primitive assembly groups vertices into a triangle. Rasterization determines which framebuffer samples the projected triangle covers. For each covered sample it creates a fragment and interpolates varying values such as color, texture coordinates, or depth.

The fragment shader computes a candidate color and possibly depth. Fragment tests and operations decide whether the candidate passes depth, stencil, scissor, or other rules, and blending may combine it with the destination color. If the write is accepted, the appropriate framebuffer attachments are updated. During presentation, the color buffer becomes the visible image.

This is the chapter benchmark: every future lecture should plug into this chain. Transformations explain the coordinate changes. Projection explains the view-to-clip step. Clipping explains view-volume boundaries. Rasterization explains sample coverage. Visibility explains depth and occlusion. Illumination, texturing, and shadows explain how fragment colors are computed.

Concrete example

A complete cube frame in the starter project follows this same path 12 times for the cube's triangles: interleaved position/color bytes -> VAO attribute interpretation -> vertex shader with uMVP -> projected triangles -> rasterized

fragments -> interpolated colors -> depth-tested framebuffer writes -> swapped display buffer.

Check yourself

Tell the full story of one vertex and one fragment in the same triangle. What data does each one carry, and when can each one be rejected or changed?

Minimum concept map

```
CPU scene decisions -> vertex records -> GPU buffers -> VAO attribute interpretation ->
vertex shader -> clip space -> clipping -> perspective division -> viewport transform ->
primitive assembly/rasterization -> fragments with interpolated values -> fragment shader
-> fragment tests and operations -> framebuffer attachments -> displayed color buffer
```

A useful exam and debugging habit is to ask three questions at every step: What representation enters? What operation changes it? What representation leaves?